



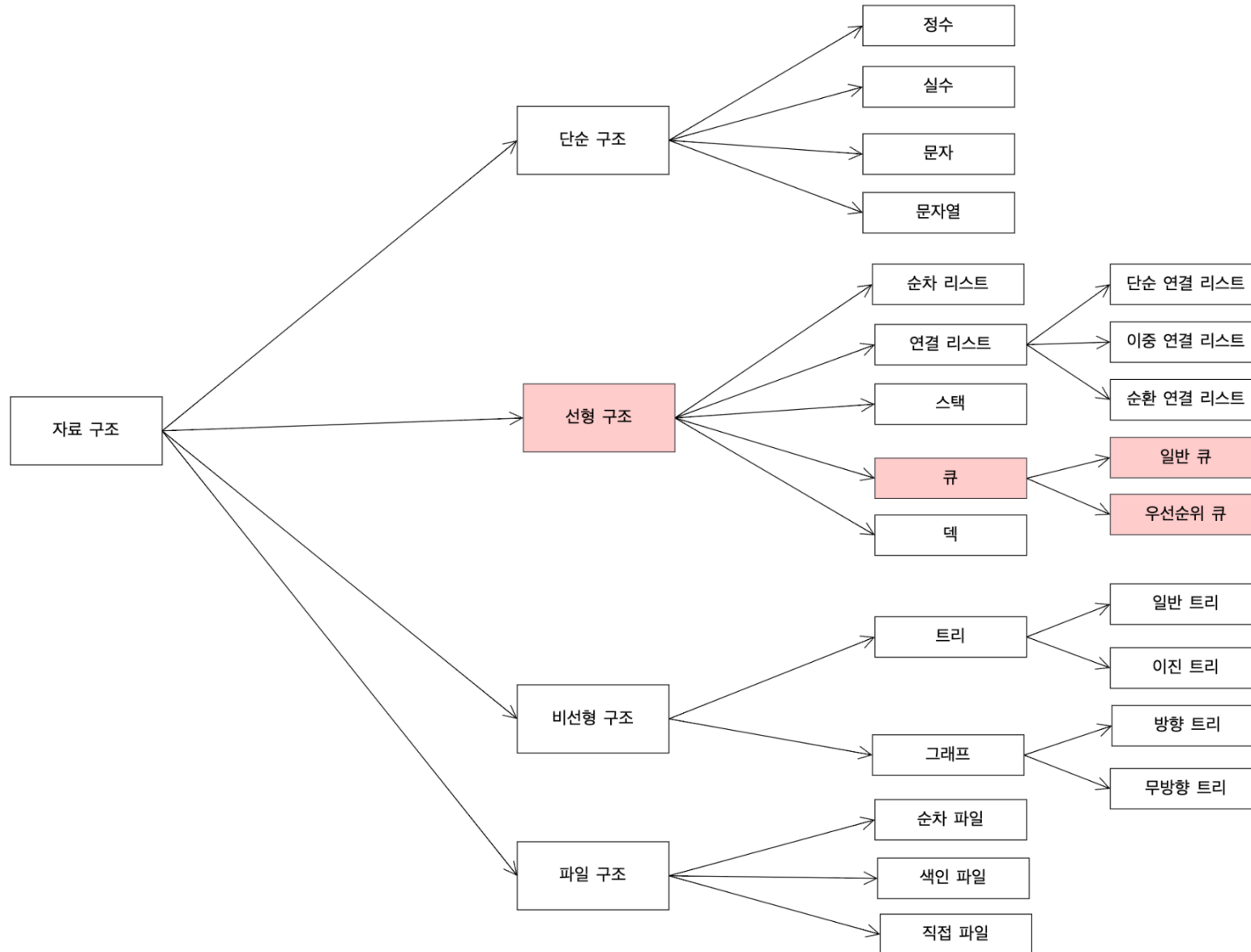
국립 한국해양대학교
NATIONAL
KOREA MARITIME & OCEAN UNIVERSITY

친환경스마트조선기자재학과

데이터처리특론



Data structure

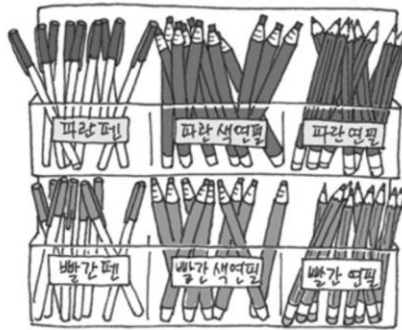


자료구조

- 데이터를 저장, 조직, 관리하는 방법



[나쁜 자료구조]



[좋은 자료구조]



자료구조 적용 전(왼쪽)과 자료구조 적용 후(오른쪽)



자료구조의 일상 예

자료구조

- 문제 해결에 사용할 부품

건축물을 만들려면

- 건축 재료와 구조 모듈에 대한 이해가 필요하다
- 철근, 시멘트, 강화 유리, 벽돌, ...
- 샷시, 철골, 거푸집, 배수 구조, 전기/인터넷 연결 구조, ...



다양한 형태의 건축물과 건축 재료

프로그래밍과 문제 해결도

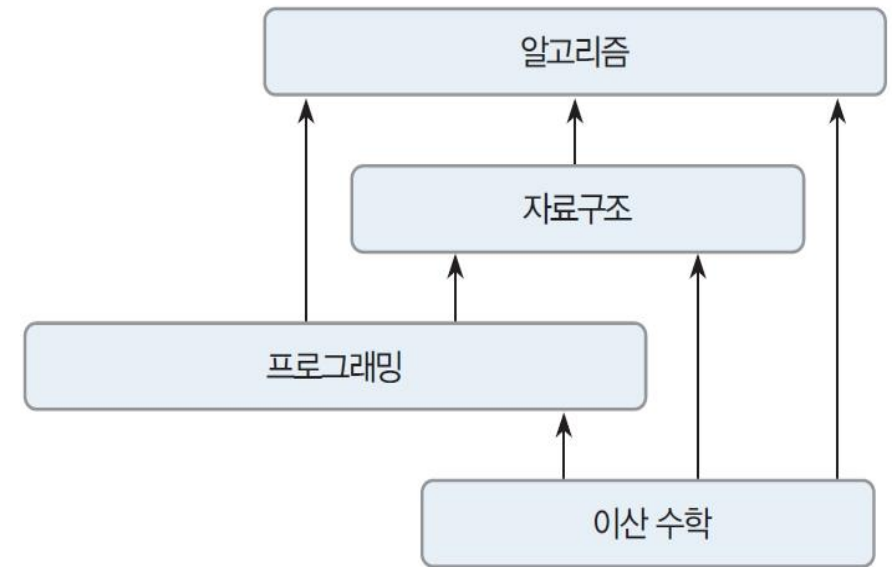
- 데이터와 구조 모듈에 대한 이해가 필요하다
- 프로그래밍 언어, 정수, 문자열, ...
- 리스트, 스택, 큐, 우선순위 큐, 검색 트리, 해시 테이블, 그래프, ...



부품(자료구조) 선택의 중요성

자료구조

- 컴퓨터 과학에서 효율적인 접근 및 수정을 가능케 하는 자료의 조직, 저장, 관리를 의미한다.
- 더 정확히 말해, 자료 구조는 데이터 값의 모임, 또 데이터 간의 관계, 그리고 데이터에 적용할 수 있는 함수나 명령을 의미한다.
- 신중히 선택한 자료구조는 보다 효율적인 알고리즘을 사용할 수 있게 한다.
- 생각하는 방법을 훈련하는 도구
- 자료구조는 그 자체로 중요하다
- 그리고 생각하는 방법을 훈련하는 것도 중요하다
 - 자료구조를 이용해서 문제를 해결하는 과정
 - 문제 해결 과정에서 논리의 골격이 구성되는 방법/스타일
 - 의미의 단위(의미의 매듭)를 설정하는 방법



알고리즘, 자료구조, 프로그래밍, 이산 수학의 관계

자료구조

- 컴퓨터에서 자료를 정리하고 조직화하는 다양한 구조
- 생활 속의 다양한 사물과 관련된 자료구조의 예



배달할 선물 목록: 리스트(List)



지하철 노선도: 그래프(Graph)



접시나 물건 쌓기: 스택(Stack)

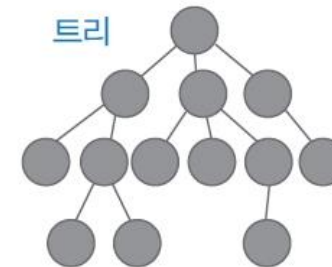
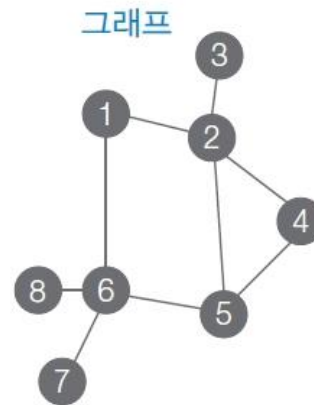
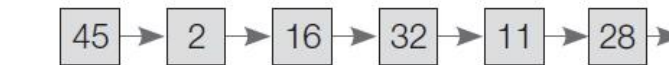
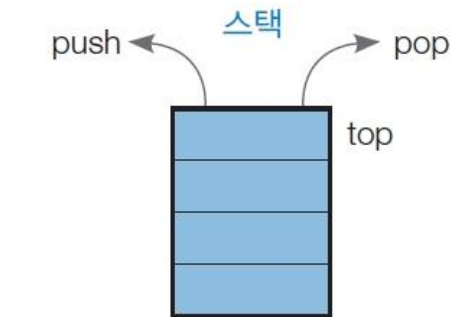
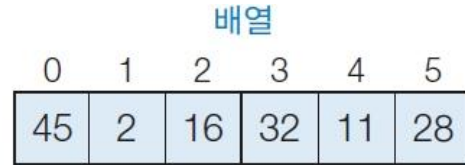


매표소 줄서기: 큐(Queue)



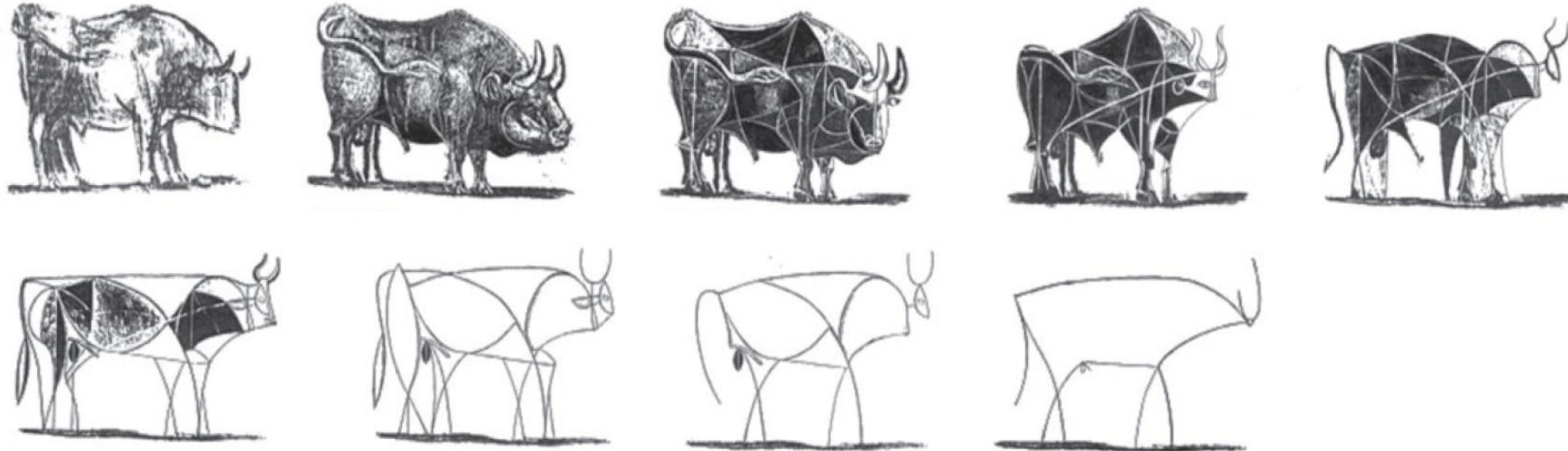
직장의 조직도: 트리(Tree)

자료구조의 종류



추상

- 세세한 부분을 표현하지 않고 전체적인 이미지를 표현한 것



추상적 묘사

Genetic Algorithm and Graph Partitioning

T. N. Bui, B. Moon • Published 1996 • Computer Science • IEEE Trans. Computers

Hybrid genetic algorithms (GAs) for the graph partitioning problem are described. The algorithms include a fast local improvement heuristic. One of the novel features of these algorithms is the schema preprocessing phase that improves GAs' space searching capability, which in turn improves the performance of GAs. Experimental tests on graph problems with published solutions showed that the new genetic algorithms performed comparable to or better than the multistart Kernighan-Lin algorithm and the simulated annealing algorithm. Analyses of some special classes of graphs are also provided showing the usefulness of schema preprocessing and supporting the experimental results. Collapse

프로그래밍 언어에서의 추상화

- 프로그래밍 언어에서의 추상화는 자료 추상화, 제어 추상화의 두 가지 범주로 나뉜다.
- 자료 추상화
 - 문자열, 수, 트리와 같은 계산 될 자료의 특성을 추상화
- 제어 추상화 or 알고리즘 추상화
 - 실행 순서를 제어하는 특성을 추상화
 - 반복문, 조건문, 프로시저 호출 등의 예가 있다.
- 추상화에 포함된 정보의 양으로 추상화의 수준을 나눌 수 있다.
 - ❖ 기본적 추상화(basic abstraction)
 - 가장 국지적인(지역적인) 기계 정보를 수집한 추상화
 - ❖ 구조적 추상화(structured abstraction)
 - 보다 전역적인 정보인 프로그램의 구조에 대한 추상화
 - ❖ 단위 추상화(unit abstraction)
 - 단위 프로그램 전체 정보에 관한 추상화

자료 추상화 – 기본적 추상화(basic abstraction)

- 컴퓨터 내부의 자료 표현을 추상화한다.

예)

기억 장치의 위치 -> 변수로 추상화

int x;

float y;

자료의 값(2진수) → 10진수(정수, 실수) 자료형으로 추상화

x = 5.7;

자료 추상화 – 구조적 추상화(structured abstraction)

- 관련된 자료값의 집합을 추상화한다.
- 배열과 레코드와 같은 구조형 자료가 전형적인 구조적 추상화의 예이다.
- 형 선언문을 가지고 자료형에 새로운 이름을 부여할 수 있다.

예)

type student = record ...

자료 추상화 – 단위 추상화(unit abstraction)

- 자료의 생성과 사용에 대한 정보를 한 장소에 모아두고, 자료의 세부사항에 대한 접근을 제한하는 도구
- 자료형의 구조 변경이 프로그램 전체에 영향을 주지 않으며, 자료 구조의 세부사항을 무시한다.
- 이런 체계를 자료 캡슐화 혹은 추상 자료형이라고도 한다.
- C++, Java의 class, Modula-2의 module, Ada의 package가 그 예이다.

제어 추상화 - 기본적 추상화(basic abstraction)

- 몇 개의 기계 명령어를 모아 이해하기 쉬운 추상 구문으로 만든 것.

예)

계산과 값의 저장을 추상화한 할당문

$x := x + y$

분기문 (Fortran의 GOTO문이나 if문 등)

IF (A.GT.B) GOTO 10

제어 추상화 - 구조적 추상화(structured abstraction)

- 프로그램에서 어떤 검사된 값에 따라 분할된 명령어의 한 그룹을 수행하도록 하는 것.
- 검사 값에 따라 분할된 명령어 그룹을 수행한다.

예)

택일문

if 문과 같은 택일문, Pascal의 case문, C언어의 switch문 등

// Ada의 if 문

if x > y then

 t := x;

 x := y;

 y := t;

else

 x := x + y;

 y := y + 1;

endif;

구조화 반복문

for, repeat, while, loop-exit

// Java의 예

i = 0;

do

 i = i + 1;

while (sentence[i] = '\$');

// Ada의 예

i := 0;

loop

 i := i + 1;

 exit when sentence(i) /= '\$';

endloop;

강력한 구조 제어: 부프로그램의 사용(프로시저 혹은 서브루틴이라고도 불림)

procedure def, procedure invocation (새 활성화 환경)

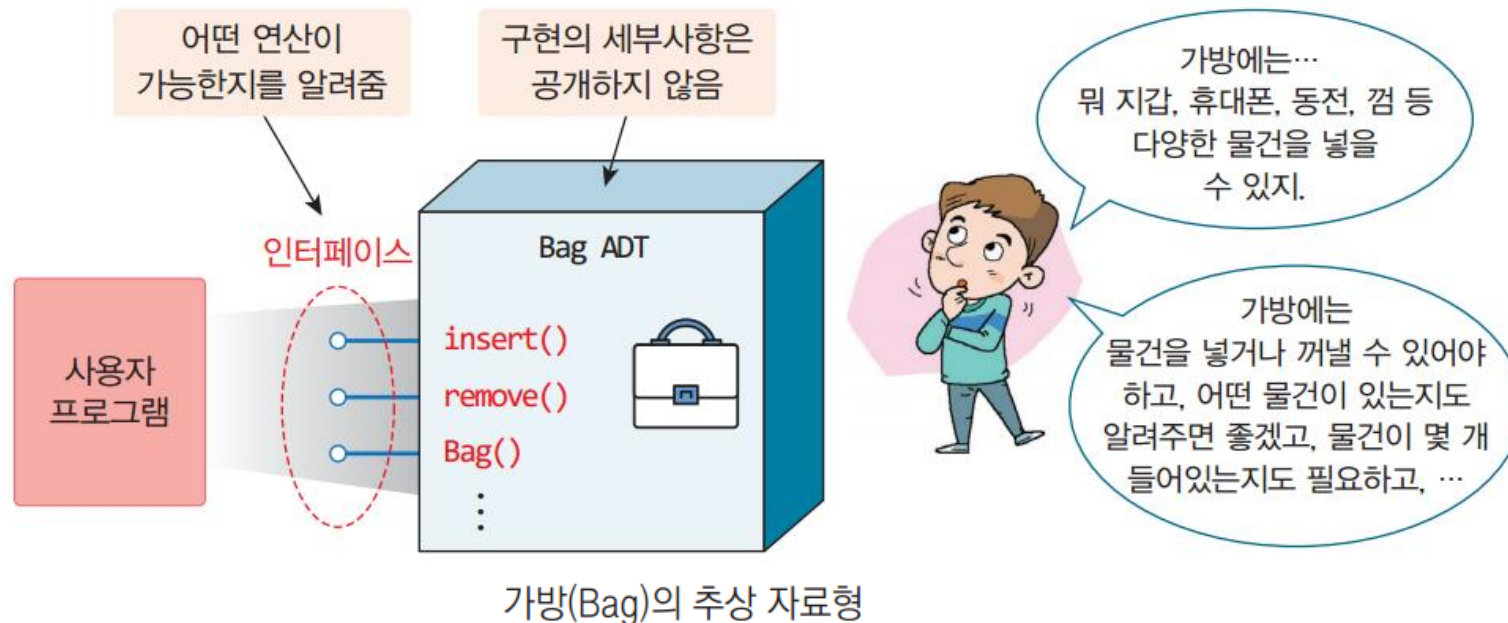
제어 추상화 - 단위 추상화(unit abstraction)

- 라이브러리처럼 프로시저의 집합을 추상화 (관련된 프로시저 그룹 추상화).
- 자료 관리 프로그램에서의 평균, 표준 편차 등 통계에 관련된 프로그램들을 모아서 하나의 단위 프로그램화가 그 예이다.
- Ada의 package, Modula-2의 module, C++와 Java의 class 등이 라이브러리 형태로 지원한다.
- 위의 범주에 들지 않는 제어 추상화로써 병행 프로그래밍이 있다.
- 많은 프로그래밍 언어가 병렬로 실행되는 프로그램들간에 동기화와 통신을 지원한다.
- Ada의 task, Modula-2의 coroutine, Java의 multithread가 이를 지원한다.

추상화는 readability(프로그램을 사람이 읽고 작성하기 쉬운 정도)의 증가가 목적이다.

추상 데이터 타입: ADT

- 세부 사항에서 벗어나 추상적으로 정의한 데이터 타입
- 즉, 어떤 데이터 타입이 어떤 작업으로 이루어지는지만 표현한 것
 - 데이터나 연산이 무엇(what)인가를 정의함
 - 데이터나 연산을 어떻게(how) 구현할 것인지는 정의하지 않음
- 시스템의 정말 핵심적인 구조나 동작에만 집중



Bag ADT

- 데이터 : 중복된 항목을 허용하는 자료들. 항목들은 특별한 순서가 없이 개별적으로 저장
- 연산
 - ✓ Bag() : 비어있는 가방을 새로 만든다.
 - ✓ Insert(item) : 가방에 항목 item을 넣는다.
 - ✓ Remove(item) : 가방에 item이 있는지 검사하여 이 항목을 꺼낸다.
 - ✓ Contains(item) : item이 들어 있으면 True를 없으면 False를 반환한다.
 - ✓ Count() : 가방에 들어 있는 항목들의 수를 반환한다.

Wireless Mouse ADT

- 데이터 : Power, LeftButton, RightButton, MouseWheel, x, y
- 연산
 - ✓ PowerOn() : 전원 On.
 - ✓ PowerOff() : 전원 Off.
 - ✓ LeftButtonClick() : 왼쪽 버튼 클릭.
 - ✓ RightButtonClick() : 오른쪽 버튼 클릭.
 - ✓ ScrollUp() : 마우스 휠 올리기
 - ScrollDown() : 마우스 휠 내리기
 - Move(x, y) : 마우스 이동하기

Bag ADT 구현(파이썬)

```
def contains(bag, e) :  
    return e in bag  
  
def insert(bag, e) :  
    bag.append(e)  
  
def remove(bag, e) :  
    bag.remove(e)  
  
def count(bag):  
    return len(bag)
```

bag에 항목 e가 있는지 검사하는 함수
파이썬의 in 연산자 사용

bag에 항목 e를 넣는 함수
파이썬 리스트의 append메소드 사용

bag에서 항목 e를 삭제하는 함수
파이썬 리스트의 remove메소드 사용

bag의 전체 항목 수를 계산하는 함수
파이썬의 len 함수 사용

Bag의 활용

- Bag을 이용한 자료 관리 예

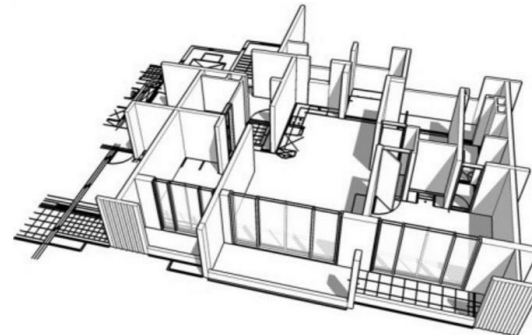
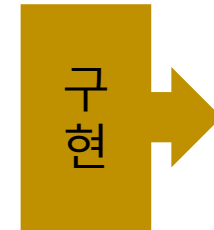
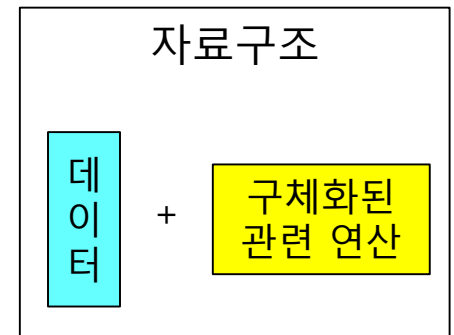
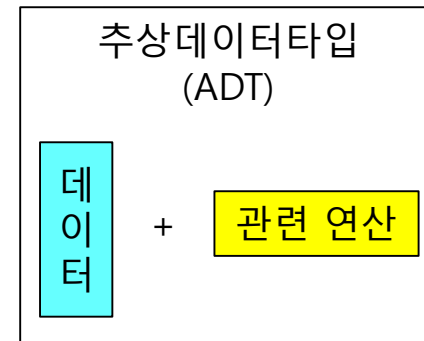
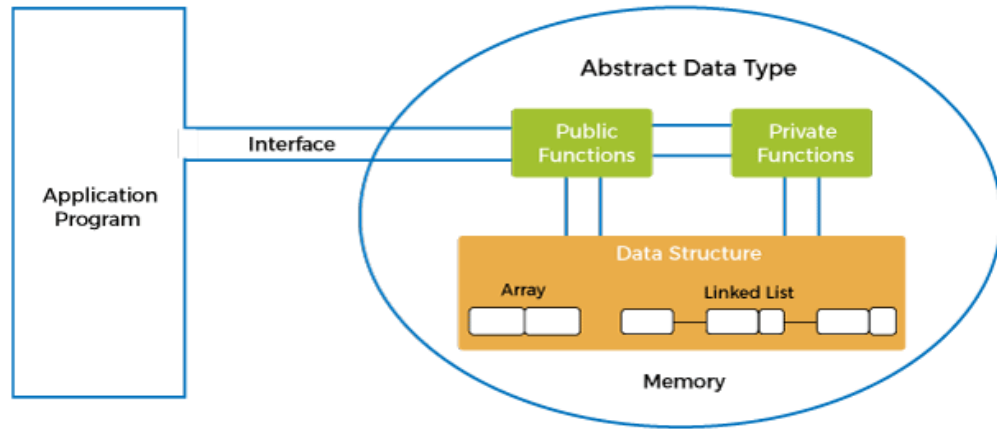
```
myBag = []                                # Bag을 위한 빈 리스트를 만들  
insert(myBag, '휴대폰')                   # Bag에 휴대폰 삽입  
insert(myBag, '지갑')                     # Bag에 지갑 삽입  
insert(myBag, '손수건')                   # Bag에 손수건 삽입  
insert(myBag, '빗')                       # Bag에 빗 삽입  
insert(myBag, '자료구조')                 # Bag에 자료구조 삽입  
insert(myBag, '야구공')                   # Bag에 야구공 삽입  
print('가방속의 물건:', myBag)            # Bag의 내용 출력  
  
insert(myBag, '빗')                       # Bag에서 '빗'삽입(중복)  
remove(myBag, '손수건')                   # Bag에서 '손수건'삭제  
print('가방속의 물건:', myBag)            # Bag의 내용 출력
```

내 가방속의 물건: ['휴대폰', '지갑', '손수건', '빗', '자료구조', '야구공']
내 가방속의 물건: ['휴대폰', '지갑', '빗', '자료구조', '야구공', '빗']

추상 데이터 타입: ADT

- 추상적 자료형은 알고리즘이 문제를 해결하는데 필요한 자료의 형태와 자료를 사용한 연산들을 수학적으로 정의한 모델이다.
- 그리고 자료구조는 추상적 자료형이 정의한 연산들을 구현한 구현체를 가리키는 말이다.
- 스택의 예를 들면, 함수 호출을 관리하기 위해 후입선출의 성질을 가진 추상적 자료형이 필요하니 pop과 push를 가지도록 스택이라는 추상적 자료형을 정의하고, 그것을 구현해서 함수 호출을 관리하는데 사용하는 구현체, 즉 자료구조를 콜 스택이라고 부르는 것이다.

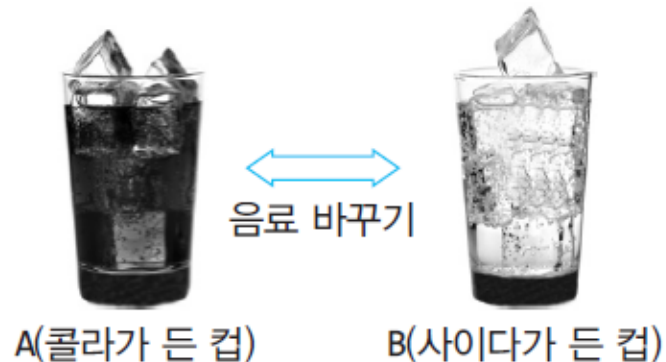
ADT & Data structure



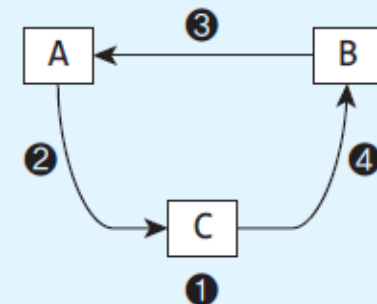
Quick Guide to Abstract Data Types

알고리즘(Algorithm)

- 수학과 컴퓨터과학에서 사용되는, 문제 해결 방법을 정의한 '일련의 단계적 절차'이자 어떠한 문제를 해결하기 위한 '동작들의 모임'이다.
- 계산을 실행하기 위한 단계적 규칙과 절차를 의미하기도 한다.
- 즉, 문제 풀이에 필요한 계산 절차 또는 처리 과정의 순서를 뜻한다.
- 프로그램명령어의 집합을 의미하기도 한다.
- 해결해야 할 어떤 문제가 주어졌을 때, 이 문제의 해답을 구하기 위한 절차를 순서대로 명확하게 나타낸 것



- ① 새로운 컵 C를 준비한다.
- ② A의 콜라를 모두 C에 붓는다.
- ③ B의 사이다를 A에 모두 붓는다.
- ④ C의 내용물을 B에 모두 붓는다.



알고리즘(Algorithm)

- 알고리즘은 여러 가지 방법으로 표현할 수 있다.
- 하나의 문제를 해결하기 위해 여러 가지 알고리즘이 가능하다.
- 동일한 문제에 대해 매우 다른 전략을 기반으로 알고리즘을 작성할 수 있고, 이들은 매우 다른 속도로 문제를 해결할 수 있다.
- 알고리즘의 정확한 동작을 위한 제한 조건이나 입력의 범위를 신중하게 고려해야 한다.
- 예) 유클리드 알고리즘에서는 a 가 b 보다 커야 한다.

알고리즘(Algorithm)의 조건

- 입력(well-defined inputs): 알고리즘이 입력을 받는다면 모호하지 않고 잘 정의된 입력이어야 한다. 알고리즘은 0개 이상의 입력을 갖는다.
- 출력(well-defined outputs): 출력은 명확하게 정의되어야 하며 1개 이상의 출력이 반드시 존재하여야 한다.
- 명확성(clear and unambiguous): 각 명령어의 의미는 모호하지 않고 명확해야 한다.
- 유한성(finiteness): 한정된 수의 단계 후에는 반드시 종료되어야 한다. 즉, 무한루프나 이와 유사한 상태로 끝나서는 안 된다.
- 유효성(feasible): 명령어들은 현재 실행 가능한 연산이어야 한다. 미래에 개발될 기술 등을 포함해서는 안 된다.

알고리즘(Algorithm)의 기술방법

- 알고리즘 기술 방법 네 가지
 - ① 영어나 한국어와 같은 자연어로 표현
 - ② 흐름도(flowchart)로 표현
 - ③ 유사 코드(pseudo-code) 사용
 - ④ 특정한 프로그래밍 언어(예: C언어나 파이썬 등)

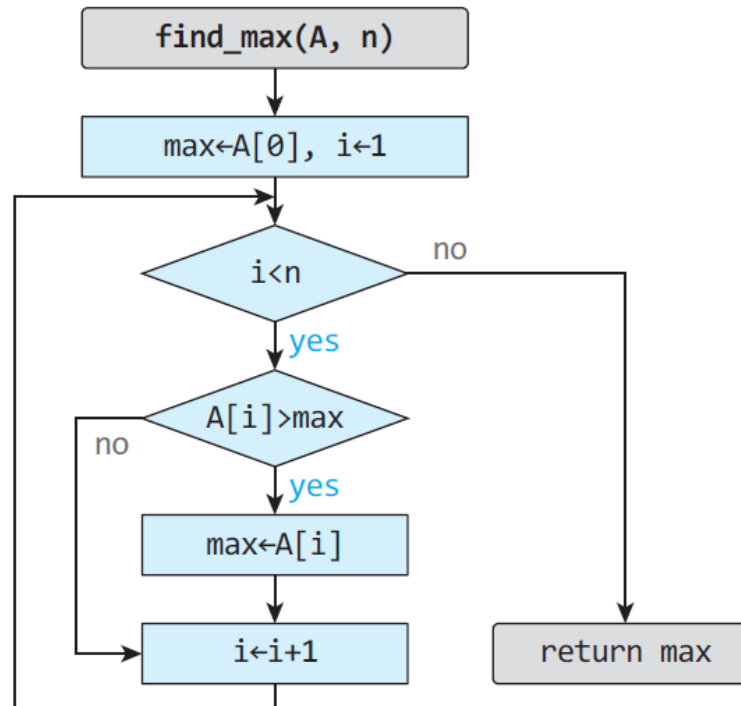
알고리즘(Algorithm)의 기술 방법

① 영어나 한국어와 같은 자연어로 표현

`find_max( A )`

1. 리스트 A의 첫 번째 항목을 변수 max에 복사한다.
2. 리스트 A의 다음 항목들을 차례대로 max와 비교하여, max보다 더 크면 그 값을 max로 복사한다.
3. 배열 A의 모든 요소를 비교했으면 max를 반환한다.

② 흐름도(flowchart)로 표현



알고리즘(Algorithm)의 기술 방법

③ 유사 코드(pseudo-code) 사용

```

01  find_max( A )
02      max←A[0]                // ← 는 대입 연산을 의미함
03      for i←1 to size(A) do
04          if A[i]>max then
05              max←A[i]
06      return max
    
```

④ 특정한 프로그래밍 언어(예: C언어)

```

01  int find_max(int A[], int n) {    // 배열의 길이도 전달해야 함
02      int i, tmp=A[0];              // 사용할 변수를 미리 선언해야 함
03      for( i=1 ; i<n ; i++ ) {      // 블록의 시작을 '{'로 표시함
04          if( A[i] > tmp ) {
05              tmp = A[i];            // =를 대입 연산자로 사용하고,
06          }                        // 비교 연산자로는 ==를 사용함
07      }                            // 중괄호 { }를 맞추어 주어야 함
08      return tmp;
09  }
    
```

알고리즘의 기술 : 파이썬

유사 코드 표현과 매우 유사함

```
01 def find_max( A ):
02     max = A[0]
03     for i in range(len(A)) :
04         if A[i] > max :
05             max = A[i]
06     return max
```

```
01 find_max( A )
02     max←A[0]
03     for i←1 to size(A) do
04         if A[i] > max then
05             max←A[i]
06     return max
```

알고리즘(Algorithm)

세 정수의 최댓값 구하기 - (1)

■ 실습 1-1

- a, b, c에 정숫값을 입력 받아 비교하고 maximum으로 최댓값을 구하는 프로그램

■ 순차 구조 sequential structure

- 한 문장씩 순서대로 처리되는 구조

■ 선택 구조 select structure

- 조건식으로 평가한 결과에 따라 프로그램의 실행 흐름이 변경되는 구조

Do it! 실습 1-1

• 완성 파일 chap01/max3.py

```

01: # 세 정수를 입력받아 최댓값 구하기 — '#' 이후의 글자는 주석으로 처리됩니다.
02:
03: print('세 정수의 최댓값을 구합니다.')
04: a = int(input('정수 a의 값을 입력하세요.: '))
05: b = int(input('정수 b의 값을 입력하세요.: '))
06: c = int(input('정수 c의 값을 입력하세요.: '))
07:
08: maximum = a
09: if b > maximum: maximum = b
10: if c > maximum: maximum = c
11:
12: print(f'최댓값은 {maximum}입니다.')
```

▶ 실행 결과

```

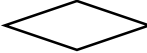
세 정수의 최댓값을 구합니다.
정수 a의 값을 입력하세요.: 1
정수 b의 값을 입력하세요.: 3
정수 c의 값을 입력하세요.: 2
최댓값은 3입니다.
```

알고리즘(Algorithm)

세 정수의 최댓값 구하기 - (2)

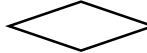
■ 세 정수의 최댓값을 구하는 알고리즘 순서도

- 알고리즘의 흐름은 위에서 아래 방향

-  : 작업 실행
-  : 조건식의 평가 결과에 따라 '예'나 '아니요' 중 하나를 선택

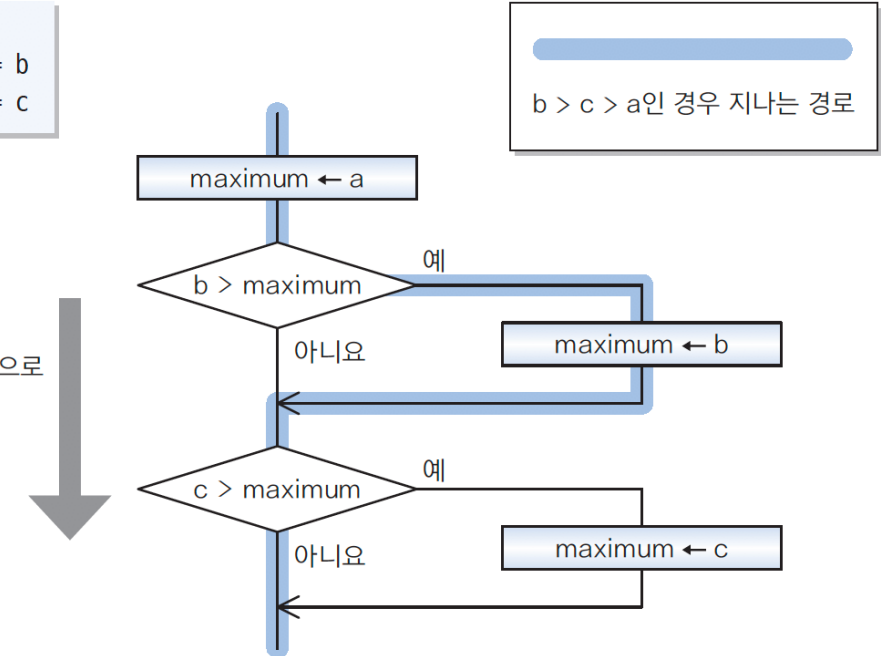
- 알고리즘(또는 프로그램)이 흐르는 방향은 조건식이 결정

■ 양 갈래 선택

-  안에 작성한 조건식에 따라 알고리즘 흐름이 두 갈래로 나뉘는 것

```
maximum = a
if b > maximum: maximum = b
if c > maximum: maximum = c
```

위에서 아래 방향으로
진행합니다.



b > c > a인 경우 지나는 경로

※ 화살표(←)는 값의 대입을 뜻한다.

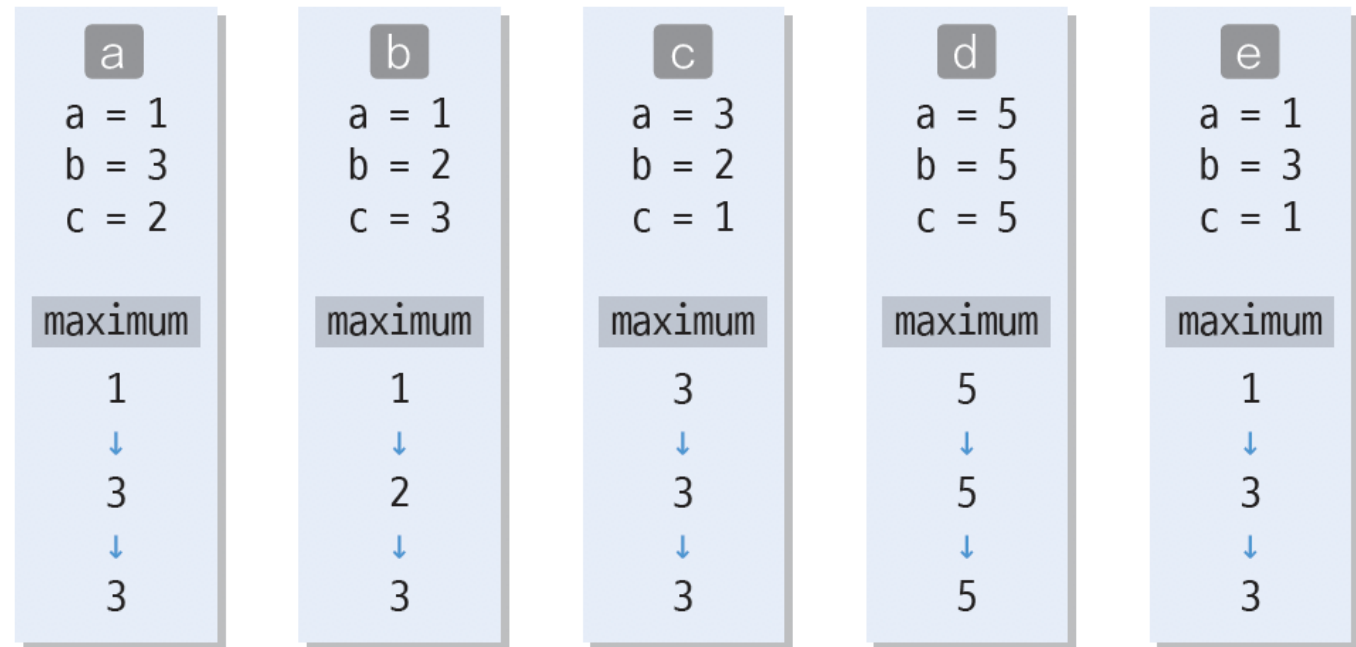
알고리즘(Algorithm)

세 정수의 최댓값 구하기 - (3)

- 세 정수의 최댓값을 구하는 과정에서 maximum값의 변화

```

maximum = a
if b > maximum: maximum = b
if c > maximum: maximum = c
    
```



알고리즘(Algorithm)

세 정수의 최댓값 구하기 - (4)

- 알고리즘의 정의

알고리즘: 어떠한 문제를 해결하기 위해 정해 놓은 일련의 절차

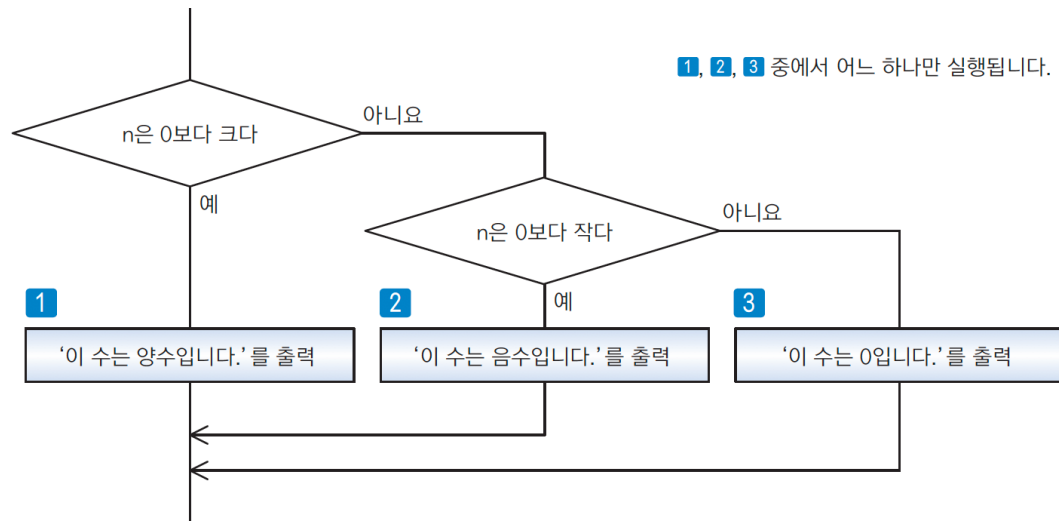
- 올바른 알고리즘이란 '어떠한 경우에도 실행 결과가 똑같이 나오는 것'을 말한다.
만약 알고리즘의 실행 결과가 어떤 경우에는 맞고 어떤 경우에는 틀리면 올바른 알고리즘이라고 할 수 없다.

알고리즘(Algorithm)

조건문과 분기 - (1)

■ 실습 1-3

- 입력 받은 정숫값의 부호(양수, 음수, 0)를 판단하여 출력하는 프로그램
- n이 양수면 **1** 이, 음수면 **2** 가, 0이면 **3** 이 실행됨
- 흐름이 3개로 분기됨



Do it! 실습 1-3

• 완성 파일 chap01/judge_sign.py

```

01: # 입력받은 정수의 부호(양수, 음수, 0) 출력하기
02:
03: n = int(input('정수를 입력하세요.: '))
04:
05: if n > 0:
06:     print('이 수는 양수입니다.') 1
07: elif n < 0:
08:     print('이 수는 음수입니다.') 2
09: else:
10:     print('이 수는 0입니다.') 3
  
```

▶ 실행 결과

```

정수를 입력하세요.: 17
이 수는 양수입니다.
정수를 입력하세요.: -5
이 수는 음수입니다.
정수를 입력하세요.: 0
이 수는 0입니다.
  
```

알고리즘(Algorithm)

조건문과 분기 - (2)

- 입력 받은 정숫값에 따라 정해진 알파벳 출력하는 프로그램 2개는 동일해보이지만 분기 개수가 다르다.

실습 1-4 : 분기 3개

Do it! 실습 1-4

• 완성 파일 chap01/branch1.py

```

01: # 3개로 분기하는 조건문
02:
03: n = int(input('정수를 입력하세요.: '))
04:
05: if n == 1:
06:     print('A')
07: elif n == 2:
08:     print('B')
09: else:
10:     print('C')
    
```

▶ 실행 결과

```

정수를 입력하세요.: 3
C
정수를 입력하세요.: 4
C
    
```

1, 2가 아니면 모두 C를 출력
→ 흐름이 3개로 분기

실습 1-5 : 분기 4개

Do it! 실습 1-5

• 완성 파일 chap01/branch2.py

```

01: # 4개로 분기하는 조건문
02:
03: n = int(input('정수를 입력하세요.: '))
04:
05: if n == 1:
06:     print('A')
07: elif n == 2:
08:     print('B')
09: elif n == 3:
10:     print('C')
    
```

▶ 실행 결과

```

정수를 입력하세요.: 3
C
정수를 입력하세요.: 4
    
```

1, 2, 3이 아니면
아무것도 출력하지 않음.
→ 흐름이 4개로 분기

알고리즘(Algorithm)

조건문과 분기 – (3)

- 실습 1-6 : 실습 1-5를 조금 더 분명하게 보이도록 재구성

Do it! 실습 1-6

• 완성 파일 chap01/branch2a.py

```
01: # 실습 1-5의 원래 모습
02:
03: n = int(input('정수를 입력하세요.'))
04:
05: if n == 1:
06:     print('A')
07: elif n == 2:
08:     print('B')
09: elif n == 3:
10:     print('C')
11: else :
12:     pass
```

※ pass 문은 '아무것도 수행하지 말고 그냥 지나치세요'를 뜻하는 키워드이다.

▶ 실행 결과

정수를 입력하세요.: 3

C

정수를 입력하세요.: 4 — else 문의 pass 문이 실행됩니다.

알고리즘(Algorithm)

순서도 기호 살펴보기 - (1)

■ 순서도 flowchart

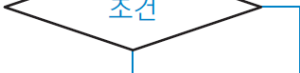
- 문제를 정의 · 분석하고 해결하는 방법을 표현한 그림
- 사용되는 기호

- 실제로 실행할 연산을 나타내는 기호
- 제어 흐름을 나타내는 선 기호
- 프로그램 순서도를 이해하거나 작성하는 데 편리한 특수 기호

용어	의미	기호
데이터	기억장치를 지정하지 않은 데이터 자체	
처리	- 여러 종류의 처리 기능을 나타냄 - 정보의 값 · 형 · 위치를 바꾸도록 정의한 연산이나 연산 집합의 실행 - 연속하는 몇 가지 흐름 가운데 하나의 방향을 결정하는 연산이나 연산 집합의 실행	

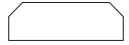
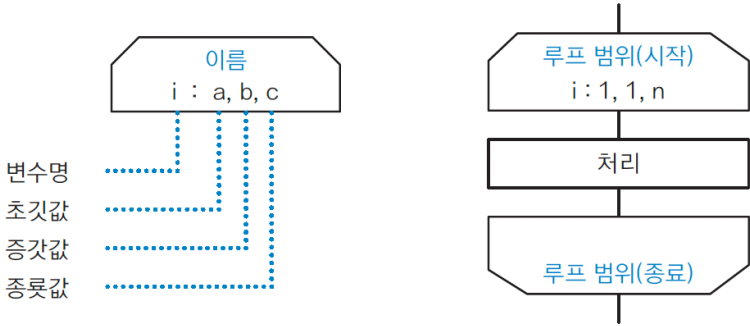
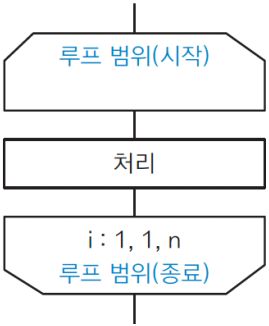
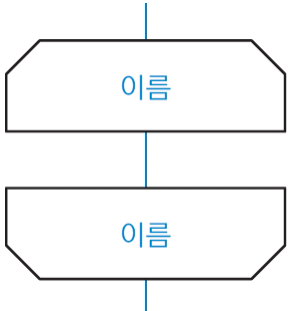
알고리즘(Algorithm)

순서도 기호 살펴보기 - (2)

용어	의미	기호
미리 정의한 처리	서브루틴이나 모듈 등 다른 곳에서 이미 정의한 하나 이상의 연산 또는 명령으로 이루어진 처리를 나타냄	
판단	- 하나의 입구와 하나 이상을 선택하는 출구가 있고, 판단 기호 안에 정의한 조건을 평가하여 하나의 출구를 선택하는 판단 기능(스위치형 기능)을 나타냄 - 주로 예상되는 평가 결과는 경로를 나타낸 선 가까이에 표기	
선	- 제어의 흐름 - 흐름의 방향을 분명히 나타낼 때는 화살표를 사용	
단말	- 외부 환경으로 나가거나 외부 환경에서 들어오는 것을 나타냄 - 주로 프로그램 흐름의 시작과 종료를 나타냄	

알고리즘(Algorithm)

순서도 기호 살펴보기 - (3)

용어	의미	기호
루프 범위	- 두 부분으로 구성되어 루프의 시작과 종료를 나타냄 2개의 루프 범위 기호 안에는 같은 이름(내용)을 표기 - 시작 기호() 또는 종료 기호() 안에 초깃값(1), 증감값(1), 종료값(n, 또는 종료 조건)을 표기 a 사전 판단 반복  b 사후 판단 반복 	

알고리즘(Algorithm)

1부터 n까지 정수의 합 구하기 - (1)

while 문 반복 알아보기

- 반복 구조 repetition structure 또는 루프 loop
 - 어떤 조건이 성립하는 동안 반복해서 처리 (프로그램 명령문 또는 명령어의 집합)
- 판단 반복 구조
 - while 문이 실행하기 전에 반복을 계속할 것인지 판단하는 구조
- 조건식의 평가 결과가 참인 동안 프로그램의 명령문이 반복됨

while 조건식: 명령문 → 루프 본문 (반복 대상)

실습 1-7

- 1부터 n까지 정수의 합을 구하는 프로그램

Do it! 실습 1-7

• 완성 파일 chap01/sum1ton_while.py

```

01: # 1부터 n까지 정수의 합 구하기 1(while 문)
02:
03: print('1부터 n까지 정수의 합을 구합니다.')
04: n = int(input('n값을 입력하세요.: '))
05:
06: sum = 0
07: i = 1
08:
09: while i <= n:    # i가 n보다 작거나 같은 동안 반복
10:     sum += i    # sum에 i를 더함
11:     i += 1     # i에 1을 더함
12:
13: print(f'1부터 {n}까지 정수의 합은 {sum}입니다.')
```

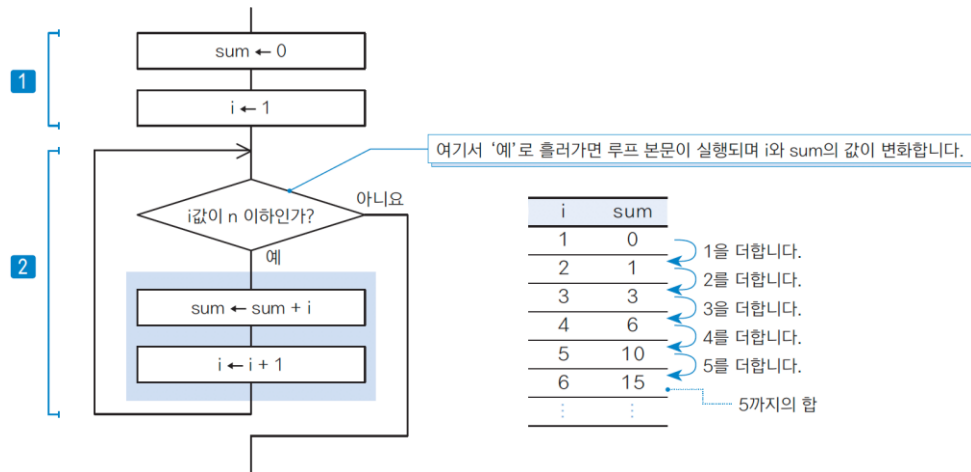
▶ 실행 결과

1부터 n까지 정수의 합을 구합니다.
 n값을 입력하세요.: 5
 1부터 5까지 정수의 합은 15입니다.

알고리즘(Algorithm)

1부터 n까지 정수의 합 구하기 - (2)

■ 실습 1-7의 프로그램 순서도



- 1은 합을 구하는 준비 단계
 - 합을 저장하는 sum값을 0으로, 반복을 제어하는 i값을 1로 초기화
- 2는 루프 본문
 - i값이 n값 이하이면 i값을 1만큼 증가시키면서 루프 본문을 실행하고, 이 과정을 n번 반복 실행

Do it! 실습 1-7

• 완성 파일 chap01/sum1ton_while.py

```

01: # 1부터 n까지 정수의 합 구하기 1(while 문)
02:
03: print('1부터 n까지 정수의 합을 구합니다.')
04: n = int(input('n값을 입력하세요.: '))
05:
06: sum = 0
07: i = 1
08:
09: while i <= n:    # i가 n보다 작거나 같은 동안 반복
10:     sum += i    # sum에 i를 더함
11:     i += 1     # i에 1을 더함
12:
13: print(f'1부터 {n}까지 정수의 합은 {sum}입니다.')
```

▶ 실행 결과

```

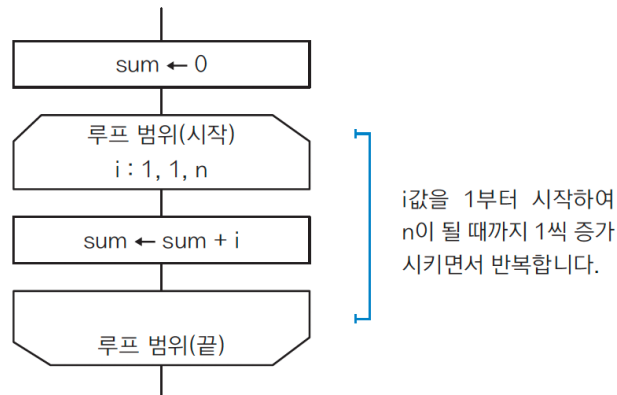
1부터 n까지 정수의 합을 구합니다.
n값을 입력하세요.: 5
1부터 5까지 정수의 합은 15입니다.
```

※ 반복을 제어할 때 사용하는 i를 **카운터용 변수**라고 한다.

알고리즘(Algorithm)

1부터 n까지 정수의 합 구하기 - (3)

- for 문 반복 알아보기
 - 변수가 하나만 있을 때는 while 문보다 for 문을 사용하는 것이 좋음
- 실습 1-8
 - 1부터 n까지 정수의 합을 for 문으로 구하는 프로그램
- 실습 1-8의 순서도



Do it! 실습 1-8

• 완성 파일 chap01/sum1ton_for.py

```

01: # 1부터 n까지 정수의 합 구하기 2(for 문)
02:
03: print('1부터 n까지 정수의 합을 구합니다.')
04: n = int(input('n값을 입력하세요.: '))
05:
06: sum = 0
07: for i in range(1, n + 1):
08:     sum += i # sum에 i를 더함
09:
10: print(f'1부터 {n}까지 정수의 합은 {sum}입니다.')
    
```

▶ 실행 결과

1부터 n까지 정수의 합을 구합니다.
 n값을 입력하세요.: 5
 1부터 5까지 정수의 합은 15입니다.

알고리즘(Algorithm)

연속하는 정수의 합을 구하기 위해 값 정렬하기

■ 실습 1-9

- 두 정수 a와 b를 오름차순으로 정렬한 다음 해당 범위의 모든 정수를 더하는 프로그램
- a가 b보다 크면 둘을 교환하여 a와 b를 오름차순으로 정렬
 - 실행 결과 1에서는 a와 b를 교환하지 않음
 - 실행 결과 2에서는 a와 b를 교환하여 출력
- a와 b 교환 시 단일 대입문 사용

```
08:     a, b = b, a # a와 b의 값을 교환(단일 대입문 사용)
```

Do it! 실습 1-9

• 완성 파일 chap01/sum.py

```
01: # a부터 b까지 정수의 합 구하기(for 문)
02:
03: print('a부터 b까지 정수의 합을 구합니다.')
04: a = int(input('정수 a를 입력하세요.: '))
05: b = int(input('정수 b를 입력하세요.: '))
06:
07: if a > b:
08:     a, b = b, a # a와 b를 오름차순으로 정렬
09:
10: sum = 0
11: for i in range(a, b + 1):
12:     sum += i # sum에 i를 더함
13:
14: print(f'{a}부터 {b}까지 정수의 합은 {sum}입니다.')
```

▶ 실행 결과 1

```
a부터 b까지 정수의 합을 구합니다.
정수 a를 입력하세요.: 3
정수 b를 입력하세요.: 8
3부터 8까지 정수의 합은 33입니다.
```

▶ 실행 결과 2

```
a부터 b까지 정수의 합을 구합니다.
정수 a를 입력하세요.: 8
정수 b를 입력하세요.: 3
3부터 8까지 정수의 합은 33입니다.
```

알고리즘(Algorithm)

반복 과정에서 조건 판단하기 1 - (1)

- **실습 1-10:** a부터 b까지 정수의 합을 구하는 과정과 최종값을 출력하는 프로그램

Do it! 실습 1-10

• 완성 파일 chap01/sum_verbose1.py

```

01: # a부터 b까지 정수의 합 구하기 1
02:
03: print('a부터 b까지 정수의 합을 구합니다.')
04: a = int(input('정수 a를 입력하세요.: '))
05: b = int(input('정수 b를 입력하세요.: '))
06:
07: if a > b:
08:     a, b = b, a
09:
10: sum = 0
11: for i in range(a, b + 1):
12:     if i < b:                # i가 b보다 작으면 합을 구하는 과정 출력
13:         print(f'{i} + ', end='') 1
14:     else:                    # i가 b보다 크거나 같으면 최종값 출력을 위해 i =를 출력
15:         print(f'{i} = ', end='') 2
16:     sum += i                # sum에 i를 더함
17:
18: print(sum)
    
```

※ 문제점: for문 반복이 n번, if 문 판단도 n번 수행

※ 더하는 수가 n개이면 + 기호는 n-1개 출력

- 1 진행 중인 값: 수 뒤에 +를 출력합니다. **예** '3 + ', '4 + ', '5 + ', '6 + '
- 2 최종값: 수 뒤에 =를 출력합니다. **예** '7 = '

▶ 실행 결과

```

a부터 b까지 정수의 합을 구합니다.
정수 a를 입력하세요.: 3
정수 b를 입력하세요.: 3
3 = 3
a부터 b까지 정수의 합을 구합니다.
정수 a를 입력하세요.: 3
정수 b를 입력하세요.: 4
3 + 4 = 7
a부터 b까지 정수의 합을 구합니다.
정수 a를 입력하세요.: 3
정수 b를 입력하세요.: 7
3 + 4 + 5 + 6 + 7 = 25
    
```

알고리즘(Algorithm)

반복 과정에서 조건 판단하기 1 – (2)

- **실습 1-11:** if문의 반복을 피하기 위해, for 문 안에 있는 if 문을 별도로 두도록 실습 1-10을 개선한 프로그램

Do it! 실습 1-11

• 완성 파일 chap01/sum_verbose2.py

```

01: # a부터 b까지 정수의 합 구하기 2
02:
03: print('a부터 b까지 정수의 합을 구합니다.')
04: a = int(input('정수 a를 입력하세요.: '))
05: b = int(input('정수 b를 입력하세요.: '))
06:
07: if a > b:
08:     a, b = b, a
09:
10: sum = 0
11: for i in range(a, b):
12:     print(f'{i} + ', end='')
13:     sum += i # sum에 i를 더함
14:
15: print(f'{b} = ', end='')
16: sum += b # sum에 b를 더함
17:
18: print(sum)
    
```

※ 문제점 해결: 실행 결과는 1-10과 같으나,
판단 횟수가 n번에서 0번으로 바뀌고 반복 횟수도 1번 감소함

- 1 진행 중인 값: for 문에서 a부터 b - 1까지 값 뒤에 +를 붙여 출력합니다.
- 2 최종값: b의 값 뒤에 =를 붙여 출력합니다.

알고리즘(Algorithm)

반복 과정에서 조건 판단하기 2 – (1)

- **실습 1-12:** 특정 문자를 줄바꿈 없이 연속으로 출력하는 프로그램

Do it! 실습 1-12

• 완성 파일 chap01/alternative1.py

```

01: # +와 -를 번갈아 출력하기 1
02:
03: print('+와 -를 번갈아 출력합니다.')
04: n = int(input('몇 개를 출력할까요?: '))
05:
06: for i in range(n):
07:     if i % 2:           # 홀수인 경우 - 출력
08:         print('-', end='')
09:     else:
10:         print('+', end='')   # 짝수인 경우 + 출력
11:
12: print()
    
```

▶ 실행 결과

```

+와 -를 번갈아 출력합니다.
몇 개를 출력할까요?: 12
+-+--+--+--+
    
```

for 문 수행

- i가 홀수면 '-'를 출력합니다.
- i가 짝수면 '+'를 출력합니다.

※ 2가지 문제점

1. for 문을 반복할 때마다 if 문 수행
2. 상황에 따라 유연하게 수정하기 어려움
 - i의 범위를 변경하면, 홀수와 짝수 순서가 바뀔 수 있음
 - 예) i를 1부터 n까지 1씩 증가시키고 싶다면, 홀수와 짝수의 순서가 변경되므로 for 문의 print() 함수 호출 순서를 변경해야 함

알고리즘(Algorithm)

반복 과정에서 조건 판단하기 2 - (2)

■ 실습 1-13: 실습 1-12의 문제점을 해결한 프로그램

Do it! 실습 1-13

• 완성 파일 chap01/alternative2.py

```
01: # +와 -를 번갈아 출력하기 2
02:
03: print('+와 -를 번갈아 출력합니다.')
04: n = int(input('몇 개를 출력할까요?: '))
05:
06: for _ in range(n // 2):
07:     print('+-', end='') # +-를 n // 2개의 출력
08:
09: if n % 2:
10:     print('+', end='') # n이 홀수일 때만 +를 출력
11:
12: print()
```

※ 파이썬에서는 무시하고 싶은 값을 언더스코어(_)로 표현할 수 있다.

▶ 실행 결과

```
+와 -를 번갈아 출력합니다.
몇 개를 출력할까요?: 12
+-+-+--+
```

- 1 이 짝수인 경우 출력
for 문은 '+'를 $n // 2$ 번 출력
예) n 이 12면 6번, n 이 15면 7번 출력
 n 이 홀수인 경우 출력
- 2 마지막에 '+'를 출력해야 함

a n 이 짝수인 경우 출력

몇 개를 출력할까요?: 12
+-+-+--+

b n 이 홀수인 경우 출력

몇 개를 출력할까요?: 15
+-+-+--++

1 '+'를 $n // 2$ 개 출력

2 '+'를 출력

※ 문제점 해결

1. 반복문에서 if 문을 수행하지 않아 효율적
2. 카운터용 변수를 0에서 1로 변경해도 유연하게 대응 가능

알고리즘(Algorithm)

반복 과정에서 조건 판단하기 3

- 실습 1-14: *을 n개 출력하되 w개마다 줄바꿈을 하는 프로그램

Do it! 실습 1-14
• 완성 파일 chap01/print_stars1.py

```

01: # *를 n개 출력하되 w개마다 줄바꿈하기 1
02:
03: print('*를 출력합니다.')
04: n = int(input('몇 개를 출력할까요?: '))
05: w = int(input('몇 개마다 줄바꿈할까요?: '))
06:
07: for i in range(n):      # n번 반복
08:     print('*', end='')
09:     if i % w == w - 1:  # n번 판단
10:         print()         # 줄바꿈
11:
12: if n % w:                # 1번 판단
13:     print()             # 줄바꿈
    
```

실행 결과
 *를 출력합니다.
 몇 개를 출력할까요?: 14
 몇 개마다 줄바꿈할까요?: 5

i를 1씩 증가시키면서 *를 출력하고, **1**과 **2**에서 줄 바꿈
1은 i를 w로 나눈 나머지가 w-1일 때 줄 바꿈
2는 n이 w의 배수가 아닐 때 마지막 줄 바꿈

a n이 15인 경우

0	1	2	3	4	* * * * *	←	1 출력 후 줄바꿈
5	6	7	8	9	* * * * *	←	
10	11	12	13	14	* * * * *	←	

b n이 14인 경우

0	1	2	3	4	* * * * *	←	1 출력 후 줄바꿈
5	6	7	8	9	* * * * *	←	
10	11	12	13	* * * *	←	2 출력 후 줄바꿈	

※ 문제점: for문 반복이 n번, if 문 판단도 n번 수행

알고리즘(Algorithm)

양수만 입력 받기

■ 실습 1-16

- 양수만 입력 받아 1부터 n까지 정수의 합을 구하는 프로그램

■ 무한 루프와 break 문 알아보기

- 무한 루프 infinite loop
 - 무한 반복되는 반복문
 - while 문의 조건식에 True를 사용하여 프로그래머가 의도적으로 while 문이 무한 반복되도록 설정
- break 문
 - 반복문 안에서 break 문을 실행하면 반복문을 종료할 수 있음
 - n이 양수인 경우 break 문이 실행되며 무한 루프에서 빠져나옴

Do it! 실습 1-16

• 완성 파일 chap01/sum1ton_positive.py

```

01: # 1부터 n까지 정수의 합 구하기(n값은 양수만 입력받음)
02:
03: print('1부터 n까지 정수의 합을 구합니다.')
04:
05: while True:
06:     n = int(input('n값을 입력하세요.: '))
07:     if n > 0:
08:         break # n이 0보다 커질 때까지 반복
09:
10: sum = 0
11: i = 1
12:
13: for i in range(1, n + 1):
14:     sum += i # sum에 i를 더함
15:     i += 1 # i에 1을 더함
16:
17: print(f'1부터 {n}까지 정수의 합은 {sum}입니다.')
```

▶ 실행 결과

```

1부터 n까지 정수의 합을 구합니다.
n값을 입력하세요.: -6 0 이하 값이면 다시 입력받습니다.
n값을 입력하세요.: 0
n값을 입력하세요.: 10
1부터 10까지 정수의 합은 55입니다.
```

알고리즘(Algorithm)

직사각형 넓이로 변의 길이 구하기

■ 실습 1-17

- 변의 길이와 넓이가 모두 정수인 직사각형에서 변의 길이를 구하는 프로그램
- 짧은 변과 긴 변의 길이는 구별하지 않음
 - 예) 직사각형의 넓이가 32이면
변의 길이를 1 × 32, 2 × 16, 4 × 8만 출력
- 약수를 나열하는 프로그램과 같음
 - 06행:** $i * i$ 가 area를 초과하면 for 문을 강제로 종료. i 가 가장 긴 변의 길이가 되기 때문
 - 07행:** continue 문은 area가 i 로 나누어 떨어지지 않으면 루프 본문의 나머지 부분을 건너뛰고 조건문으로 돌아가 for 문의 다음 반복으로 진행됨. area가 i 로 나누어 떨어지지 않으면, 변의 길이(약수)가 될 수 없기 때문.
 - 08행:** i 와 $area // i$ 의 값을 짧은 변, 긴 변 순서로 출력

Do it! 실습 1-17

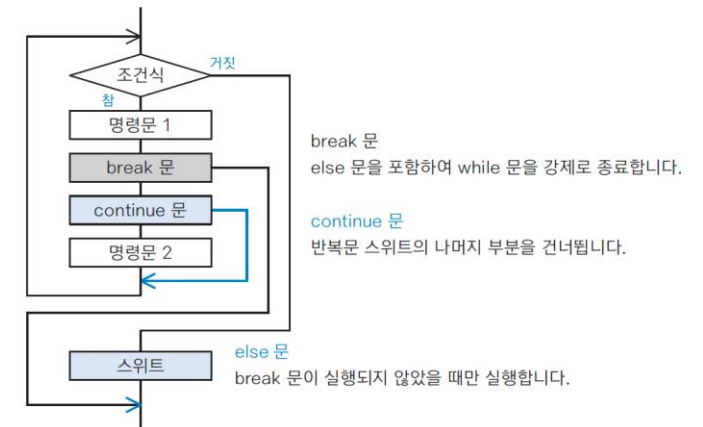
• 완성 파일 chap01/rectangle.py

```
01: # 가로, 세로 길이가 정수이고 넓이가 area인 직사각형에서 변의 길이 나열하기
02:
03: area = int(input('직사각형의 넓이를 입력하세요.: '))
04:
05: for i in range(1, area + 1): # 1부터 사각형의 넓이 계산
06:     if i * i > area: break
07:     if area % i: continue
08:     print(f'{i} × {area // i}')
```

▶ 실행 결과

```
직사각형의 넓이를 입력하세요.: 32
1 × 32
2 × 16
4 × 8
```

while 문:
명령문 1
break 문
continue 문
명령문 2
else:
스위트



알고리즘(Algorithm)

반복문 건너뛰기와 여러 범위 스캔하기

■ 실습 1-19

- 1 ~ 12까지 출력할 때 8을 건너뛰게 하는 프로그램
- 조건문과 continue 활용
- continue 문을 반복해야하는 단점이 있지만, 건너뛰어야 하는 값을 모르거나 계속 변화하는 값이라면 필요한 구현 방법

Do it! 실습 1-19

• 완성 파일 chap01/skip1.py

```

01: # 1~12까지 8을 건너뛰고 출력하기 1
02:
03: for i in range(1, 13):
04:     if i == 8:
05:         continue
06:     print(i, end=' ')
07:
08: print()
    
```

▶ 실행 결과

1 2 3 4 5 6 7 9 10 11 12

실습 1-20

건너뛰는 값을 아는 경우에 효율적인 프로그램

리스트를 활용하여, 건너뛰는 값을 제외한 범위 생성

for 문은 생성한 리스트의 원소를 하나씩 꺼내 반복하므로 반복을 위한 연산 비용은 여전히 발생

Do it! 실습 1-20

• 완성 파일 chap01/skip2.py

```

01: # 1부터 12까지 8을 건너뛰고 출력하기 2
02:
03: for i in list(range(1, 8)) + list(range(9, 13)):
04:     print(i, end=' ')
05: print()
    
```

▶ 실행 결과

1 2 3 4 5 6 7 9 10 11 12

알고리즘(Algorithm)

다중 루프 알아보기 - (1)

■ 다중 루프

- 중첩된 반복문
- 반복문이 중첩하는 정도에 따라 이중 루프, 삼중 루프 등으로 이름을 붙이고, 이를 통틀어 다중 루프라고 함

■ 실습 1-21: 이중 루프로 구구단 곱셈표를 출력하는 프로그램

- 바깥쪽 for 문은 세로 방향의 반복문으로, 각 반복은 1행, 2행, ..., 9행에 해당 (행 루프)
- 안쪽 for 문은 가로 방향의 반복문으로, 각 행에서의 1열, 2열, ..., 9열에 해당 (열 루프)

Do it! 실습 1-21

• 완성 파일 chap01/multiplication_table.py

```

01: # 구구단 곱셈표 출력하기
02:
03: print('-' * 27)
04: for i in range(1, 10):
05:     for j in range(1, 10):
06:         print(f'{i * j:3}', end='')
07:     print() # 행 변경
08: print('-' * 27)
    
```

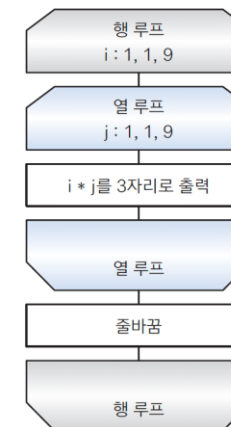
행 루프

열 루프

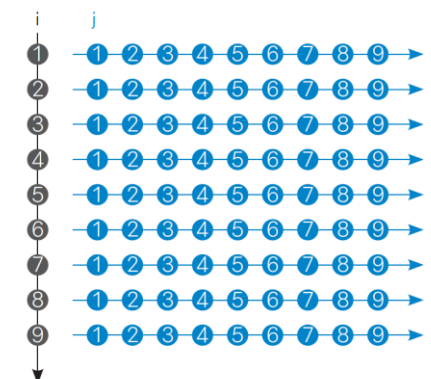
▶ 실행 결과

```

1  2  3  4  5  6  7  8  9
2  4  6  8 10 12 14 16 18
3  6  9 12 15 18 21 24 27
4  8 12 16 20 24 28 32 36
5 10 15 20 25 30 35 40 45
6 12 18 24 30 36 42 48 54
7 14 21 28 35 42 49 56 63
8 16 24 32 40 48 56 64 72
9 18 27 36 45 54 63 72 81
    
```



i와 j의 변화



알고리즘(Algorithm)

다중 루프 알아보기 - (2)

- **실습 1-22:** 왼쪽 아래가 직각인 이등변 삼각형으로 *를 출력하는 프로그램
 - 바깥쪽 for 문은 삼각형의 각 행에 대응하는 세로 방향의 반복문 (행 루프)
 - 안쪽 for 문은 삼각형의 각 열에 대응하는 가로 방향의 반복문 (열 루프)

Do it! 실습 1-22

• 완성 파일 chap01/triangle_lb.py

```
01: # 왼쪽 아래가 직각인 이등변 삼각형으로 * 출력하기
02:
03: print('왼쪽 아래가 직각인 이등변 삼각형을 출력합니다.')
04: n = int(input('짧은 변의 길이를 입력하세요.: '))
05:
06: for i in range(n):
07:     for j in range(i + 1):
08:         print('*', end='')
09:     print() # 행 변경
```

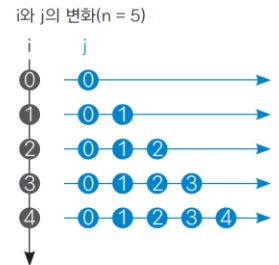
행 루프

열 루프

▶ 실행 결과

왼쪽 아래가 직각인 이등변 삼각형을 출력합니다.
짧은 변의 길이를 입력하세요.: 5

```
*
**
***
****
*****
```



※ 오른쪽 아래가 직각인 이등변 삼각형으로 *을 출력하려면, *을 출력하기 전에 공백을 출력해야 함

```
06: for i in range(n):
07:     for _ in range(n - i - 1):
08:         print(' ', end='')
09:     for _ in range(i + 1):
10:         print('*', end='')

```

공백을 출력

*을 출력

알고리즘(Algorithm)의 성능 분석

- 실행 시간을 측정하는 방법
 - 알고리즘의 실제 실행 시간을 측정하는 것
 - 알고리즘을 실제로 구현해야 함
 - 동일한 HW/SW 환경을 사용하여야 함
- 알고리즘의 복잡도를 분석하는 방법
- 직접 구현하지 않고서도 수행 시간을 분석
- 알고리즘이 수행하는 연산의 횟수를 측정하여 비교
- 연산의 횟수는 입력의 크기 n 의 함수
 - 시간 복잡도 분석 : 수행 시간 분석
 - 공간 복잡도 분석 : 필요한 메모리 공간 분석

알고리즘(Algorithm)의 성능 분석

- 자료구조의 효율성은 자료구조에 대해 수행되는 연산의 수행시간으로 측정
- 자료구조에 대한 연산 수행 시간 측정 방식은 알고리즘의 성능을 측정하는 방식과 동일
- 알고리즘의 성능: 수행 시간을 나타내는 시간 복잡도(Time Complexity)와 알고리즘이 수행되는 동안 사용되는 메모리 공간의 크기를 나타내는 공간 복잡도(Space Complexity)에 기반하여 분석
- 대부분의 경우 시간복잡도만을 사용하여 알고리즘의 성능을 분석, 주어진 문제를 해결하기 위한 대부분의 알고리즘들이 비슷한 크기의 메모리 공간을 사용하므로
- 알고리즘의 성능은 알고리즘을 구현한 프로그램을 실제로 컴퓨터에서 실행시킨 후 실행 완료까지 소요된 시간으로 측정할 수도 있다.
- 실제 측정된 시간으로 알고리즘의 성능을 객관적으로 평가하는 데는 한계가 존재한다.
- 프로그래머의 숙련도, 구현에 사용된 프로그래밍 언어의 종류 그리고 알고리즘을 실행한 컴퓨터의 성능에 따라서 수행 시간은 얼마든지 달라질 수 있기 때문

파이썬의 실행시간 측정 코드 예

01	<code>import time</code>	시각 측정 함수를 사용하기 위해 <code>time</code> 모듈을 프로그램에 포함시킴
02		
03	<code>myBag = []</code>	
04	<code>start = time.time()</code>	현재 시각을 <code>start</code> 에 저장(알고리즘 처리 전)
05	<code>insert(myBag, '축구공')</code>	실행시간을 측정하려는 코드(알고리즘)가 들어가는 부분
06	<code>...</code>	
07	<code>end = time.time()</code>	현재 시각을 <code>end</code> 에 저장(종료 시각)
08	<code>print("실행시간 = ", end-start)</code>	알고리즘의 전체 실행시간(<code>end-start</code>)

시간 복잡도(Time Complexity)

- 시간복잡도는 알고리즘(연산)이 실행되는 동안에 사용된 기본적인 연산 횟수를 입력 크기의 함수로 나타낸다.
- 기본 연산(Elementary Operation)이란 탐색, 삽입이나 삭제와 같은 연산이 아닌, 데이터 간 크기 비교, 데이터 읽기 및 갱신, 숫자 계산 등과 같은 단순한 연산을 의미한다.
- 3 종류의 분석
 - 최악경우 분석(Worst-case Analysis)
 - 평균경우 분석(Average-case Analysis)
 - 최선경우 분석(Best-case Analysis)

시간 복잡도(Time Complexity)

- 일반적으로 알고리즘의 수행 시간은 최악경우 분석으로 표현
- **최악경우 분석**
 - ✓ '어떤 입력이 주어지더라도 알고리즘의 수행시간이 얼마 이상은 넘지 않는다'라는 상한(Upper Bound)의 의미
- **평균경우 분석**
 - ✓ 입력의 확률 분포를 가정하여 분석하는데, 일반적으로 균등분포(Uniform Distribution)를 가정
- **최선경우 분석**
 - ✓ 가장 빠른 수행시간을 분석
 - ✓ 최적(Optimal) 알고리즘을 찾는데 활용

등교 시간 분석

- 집을 나와서 지하철역까지는 5분, 지하철을 타면 학교까지 30분, 강의실까지는 걸어서 10분 걸린다



(a) 최선 경우



(b) 최악 경우



(c) 평균 경우

등교 시간 분석

- 집을 나와서 지하철역까지는 5분, 지하철을 타면 학교까지 30분, 강의실까지는 걸어서 10분 걸린다
- **최악 경우 분석**
 - ✓ 열차에 승차하려는 순간, 열차의 문이 닫혀서 다음 열차를 기다려야 하고 다음 열차가 10분 후에 도착한다면, 최악경우는 $5 + 10 + 20 + 10 = 45$ 분
- **평균 경우 분석**
 - ✓ 평균 시간: 대략 최악과 최선의 중간이라고 가정했을 때, 40분이 된다.
- **최선 경우 분석**
 - ✓ 집을 나와서 5분 후 지하철역에 도착하고, 운이 좋게 바로 열차를 탄 경우를 의미한다. 따라서 최선경우 시간은 $5 + 20 + 10 = 35$ 분

수행 시간의 점근 표기법

- 수행 시간은 알고리즘이 수행하는 기본 연산 횟수를 입력 크기에 대한 함수로 표현
- 이러한 함수는 다항식으로 표현되며 이를 입력의 크기에 대한 함수로 표현하기 위해 점근 표기법 (Asymptotic Notation)이 사용
- 점근적 복잡도(Asymptotic Complexity) - 입력의 크기가 충분히 클 때의 복잡도
- **O (Big-Oh)-표기법**
- **Ω (Big-Omega)-표기법**
- **Θ (Big-Theta)-표기법**

직관적 맛보기(입력의 크기: n)

알고리즘이 **기껏해야** n^2 에 비례하는 시간이 든다 $\longrightarrow O(n^2)$

알고리즘이 **적어도** n^2 에 비례하는 시간이 든다 $\longrightarrow \Omega(n^2)$

알고리즘이 **항상** n^2 에 비례하는 시간이 든다 $\longrightarrow \Theta(n^2)$

O-표기

$O(f(n))$: 빅 오 Big Oh

- 최고차항의 차수가 $f(n)$ 보다 작거나 같은 모든 함수
- 점근적 상한
- 예, $O(n)$, $O(n \log n)$, $O(n^2)$, $O(2^n)$, ...
- $O(n^2) = \{n^2, 5n^2 + 7n + 12, 100n^2 - 2n + 9, n \log n + 5n, 3n + 9, 150, \dots\}$
 - 최고차항의 차수가 2차 이하인 모든 함수
- $5n^2 + 7n + 12 \in O(n^2)$ 으로 표기해야 하나 편의상 $5n^2 + 7n + 12 = O(n^2)$ 으로 표기함

모든 $N \geq N_0$ 에 대해서 $f(N) \leq cg(N)$ 이 성립하는 양의 상수 c 와 N_0 이 존재하면, $f(N) = O(g(N))$ 이다.

Ω-표기

$\Omega(f(n))$: 빅 오메가 Big Omega

- 최고차항의 차수가 $f(n)$ 보다 크거나 같은 모든 함수
- 점근적 하한
- 예, $\Omega(n)$, $\Omega(n \log n)$, $\Omega(n^2)$, $\Omega(2^n)$, ...
- $\Omega(n^2) = \{n^2, 5n^2 + 7n + 12, 100n^2 - 2n + 9, n^3 + 5n, 7n^5 + n^3 + 9, 2^n, n!, \dots\}$
 - 최고차항의 차수가 2차 이상인 모든 함수
- $5n^3 + 12 \in \Omega(n^2)$ 으로 표기해야 하나 편의상 $5n^3 + 12 = \Omega(n^2)$ 으로 표기함

모든 $N \geq N_0$ 에 대해서 $f(N) \geq cg(N)$ 이 성립하는 양의 상수 c 와 N_0 이 존재하면, $f(N) = \Omega(g(N))$ 이다.

Θ -표기

$\Theta(f(n))$: 빅 세타 **Big Theta**

- 최고차항의 차수가 $f(n)$ 과 **동일한** 모든 함수
- 예, $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$, $\Theta(2^n)$, ...
- $\Theta(n^2) = \{n^2, 5n^2 + 7n + 12, 100n^2 - 2n + 9, \dots\}$
- 최고차항의 차수가 2차인 모든 함수
- $5n^2 + 12 \in \Theta(n^2)$ 으로 표기해야 하나 편의상 $5n^2 + 12 = \Theta(n^2)$ 으로 표기함

모든 $N \geq N_0$ 에 대해서 $c_1g(N) \geq f(N) \geq c_2g(N)$ 이 성립하는 양의 상수 c_1, c_2, N_0 가 존재하면, $f(N) = \Theta(g(N))$ 이다.

점근적 복잡도를 수학적으로 증명할 때는 이런 방식의 정의가 필요하지만 자료구조 과목 수준에서는 앞의 직관적 정의로 충분하다.

O, Ω , Θ 표기법 사용해 보기

```
alg1(A[], n):  
    k  $\leftarrow$  n/2  
    return A[k]
```

n에 관계없이 상수 시간이 소요된다
 $\Omega(1)$ 이기도 하고 $O(1)$ 이기도 하다
 $\Theta(1)$: 가장 정확한 표현

```
alg2(A[], n):  
    sum  $\leftarrow$  0  
    for i  $\leftarrow$  0 to n-1  
        sum  $\leftarrow$  sum + A[i]  
    return sum
```

항상 n에 비례하는 시간이 소요된다
 $\Omega(n)$ 이기도 하고 $O(n)$ 이기도 하다
 $\Theta(n)$: 가장 정확한 표현

O, Ω , Θ 표기법 사용해 보기

alg3(A[], n) :

sum $\leftarrow$ 0

if (n0이 홀수)

for i $\leftarrow$ 0 **to** n-1

 sum $\leftarrow$ sum + A[i]

return sum

최악의 경우 n에 비례하는 시간이 소요된다

최선의 경우 상수 시간이 소요된다

$\Omega(1)$ 이고 $O(n)$ 이다

최선의 경우 $\Theta(1)$, 최악의 경우 $\Theta(n)$ 이라고 말한다

alg4(A[], n):

sum $\leftarrow$ 0

for i $\leftarrow$ 0 **to** n-2

for j $\leftarrow$ i+1 **to** n-1

 sum $\leftarrow$ sum + A[i] * A[j]

return sum

항상 n^2 에 비례하는 시간이 소요된다

$\Omega(n^2)$ 이기도 하고 $O(n^2)$ 이기도 하다

$\Theta(n^2)$: 가장 정확한 표현

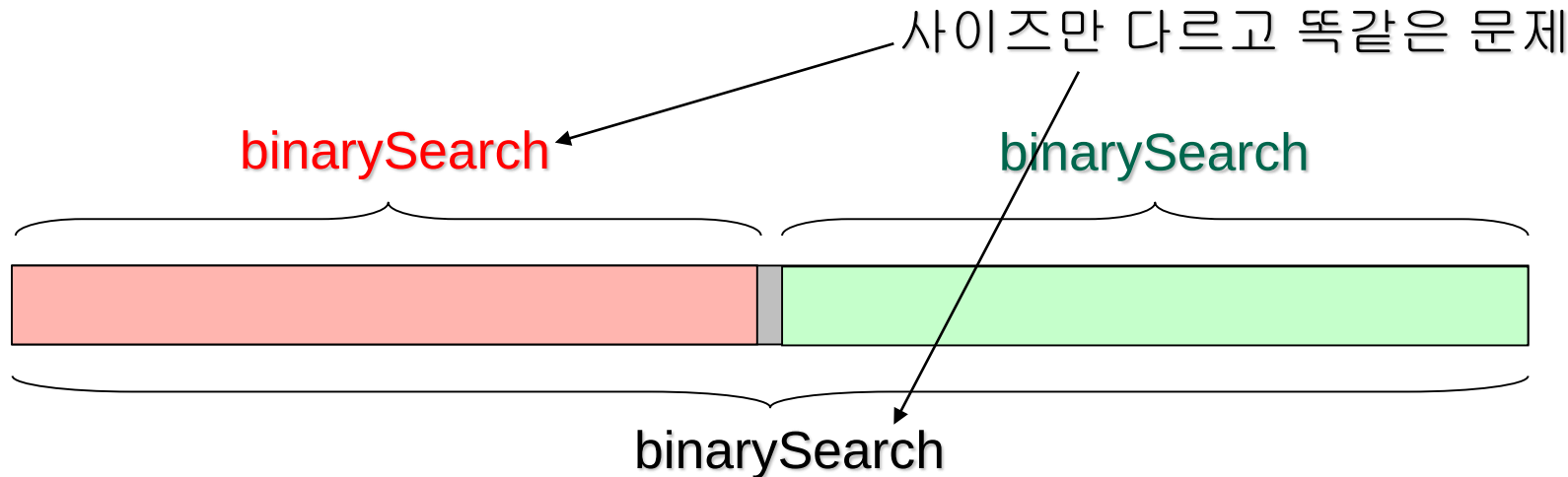
재귀 알고리즘의 복잡도 분석 예

```

binarySearch(A[], x, low, high) :
    // A: array, x: search key, low, high: array bounds
    if (low > high) return "Not found"
    mid ← (low + high)/2
    if (A[mid] < x) return binarySearch(A, x, mid+1, high)
    else if (A[mid] > x) return binarySearch(A, x, low, mid-1)
    else return mid
    
```

배열의 중앙에 있는 원소와 비교하고 나면
 자신과 똑같지만 크기가 반이 되는 문제를
 만난다. 즉, $T(n) = c + T(n/2)$
 이런 식으로 크기를 반씩 줄여나가면
 $\sim \log_2 n$ 번 만에 크기 1인 문제를 만나게 된다.
 문제의 크기를 반으로 줄이는데 필요한 작업은
 상수 시간이므로 최대 $\log_2 n$ 에 비례하는 시간에 끝난다.

수행 시간:
 최악의 경우 $\Theta(\log n)$,
 최선의 경우 $\Theta(1)$,
 앞에 수식어 없이 그냥 말하면 $O(\log n)$



Reference



- IT CookBook, 쉽게 배우는 자료구조 with 파이썬, 문병로, 한빛 아카데미
- 파이썬으로 쉽게 배우는 자료구조, 최영규, 천인국, 생능출판사
- Do it! 자료구조와 함께 배우는 알고리즘 입문 - 파이썬 편, 시바타 보요, 강민, 이지스퍼블리싱
- 파이썬과 함께하는 자료구조의 이해, 양성봉, 생능출판사
- 자료구조와 알고리즘 with 파이썬, 최영규, 생능북스
- https://ko.wikipedia.org/wiki/%EC%9E%90%EB%A3%8C_%EA%B5%AC%EC%A1%B0
- <https://velog.io/@abc2752/%EC%8B%9C%EA%B0%84-%EB%B3%B5%EC%9E%A1%EB%8F%84Time-Complexity>